

Divide y vencerás

De los ciclos con invariante a la recursión

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto de 2026

Objetivos de la sesión

Al terminar la clase, usted podrá

- Repasar los invariantes I_0 e I_1 de un ciclo y su demostración por inicialización, estabilidad y terminación (OA6).
- Reconocer el esquema dividir, conquistar y combinar de un algoritmo recursivo (OA1).
- Plantear la recurrencia $T(n) = a \cdot T(n/b) + f(n)$ de un algoritmo de divide y vencerás (OA6).
- Implementar la búsqueda del máximo y el ordenamiento por mezcla, y estimar sus costos (OA1).

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

Especificación e invariantes

Lo esencial de los invariantes de ciclo

Un problema se especifica con su **entrada** (precondiciones) y su **salida** (poscondiciones), y un algoritmo es **correcto** si calcula el resultado correcto para todas las instancias. Para los algoritmos iterativos, la herramienta es el **invariante**: una propiedad sobre las variables que debe ser cierta a lo largo de la ejecución del ciclo.

Siempre en pareja

- I_0 : las cotas del índice.
- I_1 : qué lleva el acumulador en función del índice.

El método de la demostración

El método

Inicialización: la fórmula es verdadera antes de la primera iteración.

Estabilidad: si es cierta antes de cualquier iteración, sigue siendo cierta después. **Terminación:** el ciclo debe terminar y los invariantes deben suministrar información sobre el objetivo del algoritmo.

El repaso en acción: el factorial

```
def fact(N):  
    ans = 1  
    i = 1  
    while i <= N:  
        ans = ans * i  
        i = i + 1  
    return ans
```

$N = 5$

(i, ans)

$(1, 1) \rightarrow (2, 1) \rightarrow (3, 2) \rightarrow (4, 6) \rightarrow (5, 24) \rightarrow (6, 120)$
 $0! \quad 1! \quad 2! \quad 3! \quad 4! \quad 5!$
 $1 \leq i \leq N+1$
 $ans = (i-1)!$

Teorema 1

Los invariantes $I_0 : 1 \leq i \leq N+1$ e $I_1 : ans = (i-1)!$ se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación. El desarrollo queda en el apéndice.

Otro ejercicio: la búsqueda de un valor

El problema

Entrada: un arreglo $A[0..N)$ de números y un número v . **Salida:**
 $\exists p \in [0..N). A[p] = v$.

```
0 bool solve(vector<int>& A, int v) {  
1     bool ans = false;  
2     int i = 0;  
3     while (i < A.size()) {  
4         if (A[i] == v)  
5             ans = true;  
6         ++i;  
7     }  
8     return ans;  
9 }
```

Ahora usted

Aquí `ans` no acumula un número: acumula una respuesta. Proponga I_0 e I_1 antes de mirar el apéndice.

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos**
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

¿El mayor de todos?

```
def maximo_iterativo(lista):  
    mayor = lista[0]  
    i = 1  
    while i < len(lista):  
        if lista[i] > mayor:  
            mayor = lista[i]  
        i = i + 1  
    return mayor
```

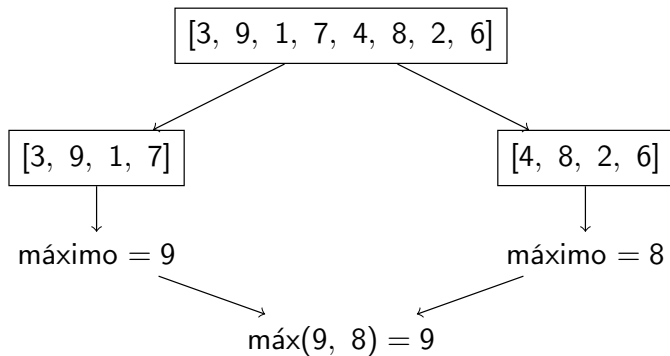
La solución que ya sabemos escribir (y certificar)

Un ciclo que recorre las n posiciones una vez, con sus dos invariantes: $I_0 : 1 \leq i \leq n$ e I_1 : $\text{mayor} = \text{máx}(\text{lista}[0..i])$, el máximo del prefijo ya recorrido.

Pregunta

¿Hay otra manera de pensar el problema, sin recorrerlo de izquierda a derecha?

Partir el problema



Partir el problema: la idea

La idea

Si alguien me dice el máximo de la mitad izquierda y el de la derecha, el máximo total es el mayor de esos dos. Y cada mitad es *el mismo problema*, más pequeño: se resuelve igual, partiendo otra vez, hasta llegar a arreglos de un solo elemento.

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás**
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

El esquema

Divide y vencerás (CLRS, Sección 2.3.1)

Un algoritmo de divide y vencerás resuelve un problema de tamaño n en tres momentos:

- 1 **Dividir** el problema en subproblemas más pequeños *del mismo tipo*. ↗ Caso base
- 2 **Conquistar** cada subproblema resolviéndolo recursivamente; si es tan pequeño que resulta trivial, resolverlo directo: ese es el **caso base**.
- 3 **Combinar** las soluciones parciales en la solución del problema original.

El esquema en el máximo

En el máximo

Dividir: partir el arreglo por la mitad. Conquistar: el máximo de cada mitad. Combinar: quedarse con el mayor de los dos. Caso base: un arreglo de un elemento es su propio máximo.

¿Y la correctitud?

Para los algoritmos recursivos la técnica es la **inducción estructural**: el caso base se verifica directo y el caso recursivo asume que las llamadas sobre problemas más pequeños son correctas. Es la pareja de los invariantes, que cubren los iterativos.

La recurrencia

El costo de un divide y vencerás

Si el problema de tamaño n se parte en a subproblemas de tamaño n/b , y dividir más combinar cuesta $f(n)$, el tiempo cumple

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n), \quad T(1) = \Theta(1).$$

- $a \geq 1$: cuántos subproblemas se generan.
- $b > 1$: en cuánto se reduce el tamaño en cada nivel.
- $f(n)$: lo que cuesta partir y luego juntar.

La recurrencia del máximo

Para el máximo

Dos mitades ($a = 2$, $b = 2$) y combinar es una sola comparación:
 $f(n) = \Theta(1)$. Queda $T(n) = 2 T(n/2) + \Theta(1)$.

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo**
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

Máximo recursivo

```
def maximo(lista, ini, fin):  
    # Maximo de lista[ini..fin] partiendo el rango en dos  
    if ini == fin:  
        resultado = lista[ini]  ← Caso base arreglo tamaño 1  
    else:  
        mitad = (ini + fin) // 2  ← Dividir  
        max_izq = maximo(lista, ini, mitad)  Menor  
        max_der = maximo(lista, mitad + 1, fin)  Mayor  
        if max_izq >= max_der:  
            resultado = max_izq  
        else:  
            resultado = max_der  ← Combinar  
    return resultado
```

Los tres momentos, en el código

Dividir es calcular mitad; conquistar son las dos llamadas recursivas; combinar es el if final. El caso base es el rango de un solo elemento.

¿Cuánto cuesta? ¿Ganamos algo?

Resolviendo $T(n) = 2 T(n/2) + \Theta(1)$ por expansión

Cada llamada sobre más de un elemento genera dos llamadas sobre la mitad. Contando llamadas: $1 + 2 + 4 + \dots$ hasta las n hojas, es decir $2n - 1$ llamadas de costo constante. Por lo tanto $T(n) \in O(n)$.

Pregunta

El ciclo también era $O(n)$. ¿Para qué sirvió entonces partir el problema?

¿Ganamos algo? Respuesta

Aquí no, y eso es normal

Para hallar el máximo hay que mirar todos los elementos al menos una vez: ningún orden de visita baja de $O(n)$. Partir el arreglo no hizo magia.

Lo que sí ganamos

Una forma de atacar el problema. Cuando las dos mitades hagan trabajo *que se aprovecha al combinar*, la cuenta cambia de verdad. Eso pasa con el problema de ordenar, que es el plato fuerte de hoy.

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla**
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

Ordenar partiendo en dos

La pregunta clave

Ordenar un arreglo grande es difícil. Pero suponga que las dos mitades ya vinieran ordenadas:

[2, 4, 7] y [1, 3, 6]. [1 2 3 4 6 7]

¿Qué tan difícil es juntarlas en una sola lista ordenada?

La respuesta que hace funcionar todo

Es fácil: el menor de todos está al frente de una de las dos listas. Se toma, y se repite. Esa operación se llama **mezcla** (*merge*), y sobre ella se construye el ordenamiento por mezcla, *mergesort* (CLRS, Sección 2.3.1, pp. 30–34).

La primera versión: mezclar dos listas

```
def mezclar(izq, der):  
    # Combina dos listas ordenadas en una lista nueva  
    resultado = []  
    i = 0  
    j = 0  
    while i < len(izq) and j < len(der):  
        if izq[i] <= der[j]:  
            resultado.append(izq[i])  
            i = i + 1  
        else:  
            resultado.append(der[j])  
            j = j + 1  
    while i < len(izq):  
        resultado.append(izq[i])  
        i = i + 1  
    while j < len(der):  
        resultado.append(der[j])  
        j = j + 1  
    return resultado
```


La primera versión: ordenar

```
def ordenar(lista):  
    # Ordena por mezcla: divide, conquista y combina  
    if len(lista) <= 1:  
        resultado = lista  
    else:  
        mitad = len(lista) // 2  
        izq = ordenar(lista[0:mitad])  
        der = ordenar(lista[mitad:len(lista)])  
        resultado = mezclar(izq, der)  
    return resultado
```

Los tres momentos

Dividir: cortar la lista por la mitad con slices (`lista[0:mitad]` produce una lista nueva con esa mitad). Conquistar: ordenar cada mitad con el mismo procedimiento. Combinar: mezclar. Caso base: una lista de cero o un elemento ya está ordenada.

La primera versión: ordenar

Funciona, y se escribe rápido

Es la traducción directa del esquema. La pregunta que sigue: ¿cuánto cuesta dividir así?

¿Cuánto cuesta dividir copiando?

Lo que cuesta un slice

`lista[a:b]` construye una lista nueva copiando sus $b - a$ elementos uno por uno: tiempo y memoria $\Theta(b - a)$. Dividir con slices convierte un paso que debía ser $\Theta(1)$ en uno que cuesta $\Theta(n)$.

¿Cuánto cuesta dividir copiando?

La cuenta, sobre los algoritmos del curso

- En el máximo, la recurrencia pasaría de $T(n) = 2 T(n/2) + \Theta(1)$ a $T(n) = 2 T(n/2) + \Theta(n)$: de $O(n)$ a $O(n \lg n)$ solo por copiar al partir.
- En la búsqueda binaria, que viene enseguida, es peor:
 $T(n) = T(n/2) + \Theta(1)$ da $O(\lg n)$, y con slices
 $T(n) = T(n/2) + \Theta(n)$ da $O(n)$: se pierde toda la gracia del algoritmo.
- En el ordenamiento por mezcla el orden se salva, porque cada nivel ya paga $\Theta(n)$ al mezclar; pero cada nivel copia otros n elementos solo para partir, y las celdas pedidas y liberadas suman $\Theta(n \lg n)$.

¿Cuánto cuesta dividir copiando?

La mejora

Dividir debe ser gratis. Los índices lo logran: cada llamado dice *de dónde a dónde* trabaja, como ya lo hace `maximo(lista, ini, fin)`. Lo que sigue es transformar esta versión, pieza por pieza.

Ordenar por índices, sobre el mismo arreglo

```
def ordenar(lista, ini, fin):  
    # Ordena lista[ini..fin] en su lugar, partiendo el rango en dos  
    if ini < fin:  
        mitad = (ini + fin) // 2  
        ordenar(lista, ini, mitad)  
        ordenar(lista, mitad + 1, fin)  
        mezclar(lista, ini, mitad, fin)  
    return lista
```

Qué cambió

Dividir ahora es calcular mitad: los índices delimitan cada mitad sin mover un elemento, $\Theta(1)$. El caso base quedó implícito: un rango de cero o un elemento ($ini \geq fin$) no entra al if. Y mezclar ya no puede devolver una lista nueva: debe dejar el resultado en `lista[ini..fin]`, el mismo tramo donde viven las dos mitades. El arreglo completo se ordena con `ordenar(lista, 0, len(lista) - 1)`.

Mezclar sobre el mismo arreglo: las copias

```
def mezclar(lista, ini, mitad, fin):  
    # Combina los tramos ordenados lista[ini..mitad]  
    # y lista[mitad+1..fin] sobre lista[ini..fin]  
    izq = []  
    der = []  
    p = ini  
    while p <= mitad:  
        izq.append(lista[p])  
        p = p + 1  
    while p <= fin:  
        der.append(lista[p])  
        p = p + 1
```

Por qué copiar los dos tramos

La mezcla escribe sobre `lista[ini..fin]`, el mismo tramo donde viven los elementos que está leyendo. Sin las copias `izq` y `der`, una escritura podría tapar un valor que todavía no ha salido. Así construye CLRS su procedimiento MERGE (p. 31), con los arreglos auxiliares *L* y *R*.

Mezclar sobre el mismo arreglo: la mezcla

```
i = 0
j = 0
k = ini
while i < len(izq) and j < len(der):
    if izq[i] <= der[j]:
        lista[k] = izq[i]
        i = i + 1
    else:
        lista[k] = der[j]
        j = j + 1
    k = k + 1
while i < len(izq):
    lista[k] = izq[i]
    i = i + 1
    k = k + 1
while j < len(der):
    lista[k] = der[j]
    j = j + 1
    k = k + 1
return lista
```


El ciclo de mezclar tiene su invariante

Los invariantes del primer ciclo

$$I_0: 0 \leq i \leq \text{len}(\text{izq}) \wedge 0 \leq j \leq \text{len}(\text{der}) \wedge k = \text{ini} + i + j$$

I_1 : lista[ini..k) contiene, en orden, los $i + j$ elementos más pequeños de las dos copias.

Los dos teoremas

Teorema 1: I_0 e I_1 se cumplen — inicialización, estabilidad (aquí por casos, según cuál copia aporta el menor) y la terminación, que al agotarse una de las dos deja escritos, en orden, los $i + j$ menores. **Teorema 2:** mezclar deja lista[ini..fin] ordenado con los mismos elementos que tenía. La escritura completa queda como ejercicio.

Mezclar: no es casualidad

No es casualidad

Así prueba CLRS la correctitud de su procedimiento MERGE (pp. 31–33): con un invariante de ciclo cuya *estabilidad* se revisa por casos, según cuál lista aporta el menor. Las dos técnicas del curso no compiten: la correctitud de la recursión se argumenta por inducción estructural, y los invariantes certifican los ciclos que quedan adentro.

Traza de la mezcla

`mezclar(lista, 0, 2, 5)` con `lista = [2,4,7,1,3,6]`

Las copias quedan `izq = [2,4,7]` y `der = [1,3,6]`.

<i>i</i>	<i>j</i>	<i>k</i>	Comparación	lista tras escribir
0	0	0	$2 > 1$: se escribe 1	[1, 4, 7, 1, 3, 6]
0	1	1	$2 \leq 3$: se escribe 2	[1, 2, 7, 1, 3, 6]
1	1	2	$4 > 3$: se escribe 3	[1, 2, 3, 1, 3, 6]
1	2	3	$4 \leq 6$: se escribe 4	[1, 2, 3, 4, 3, 6]
2	2	4	$7 > 6$: se escribe 6	[1, 2, 3, 4, 6, 6]
2	3	5	der agotada: se escribe 7	[1, 2, 3, 4, 6, 7]

Traza de la mezcla

Mire la segunda fila

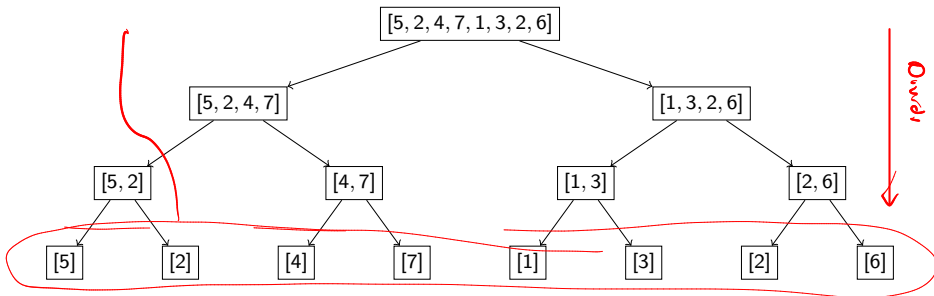
La escritura en $k = 1$ tapa el 4 cuando el 4 todavía no ha salido. La mezcla sobrevive porque lee de las copias, nunca del tramo que está escribiendo.

El costo

Cada vuelta escribe exactamente un elemento y ninguno se escribe dos veces: mezclar dos tramos con n elementos en total cuesta $\Theta(n)$, contando las copias iniciales, que son otras n operaciones.

El árbol de llamadas

Caso base



Y de regreso, mezclando

Leído de abajo hacia arriba, cada nivel se reconstruye con mezclas:

$[2, 5] [4, 7] [1, 3] [2, 6] \rightarrow [2, 4, 5, 7] [1, 2, 3, 6] \rightarrow [1, 2, 2, 3, 4, 5, 6, 7].$

Es la Figura 2.4 de CLRS (p. 35): bajar dividiendo, subir mezclando.

¿Cuánto cuesta ordenar así?

La recurrencia

Dos mitades ($a = 2$, $b = 2$) y combinar es la mezcla, que cuesta $\Theta(n)$:

$$T(n) = 2 T\left(\frac{n}{2}\right) + \Theta(n).$$

Contar el costo por niveles

Contándola por niveles (CLRS, Sección 2.3.2)

Nivel	Subproblemas	Tamaño de cada uno	Costo del nivel
0	1	n	$c \cdot n$
1	2	$n/2$	$c \cdot n$
2	4	$n/4$	$c \cdot n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\lg n$	n	1	$c \cdot n$

Todos los niveles cuestan lo mismo, $c \cdot n$, y hay $\lg n + 1$ niveles. El total es $c n(\lg n + 1)$, es decir $T(n) \in \Theta(\underline{n \lg n})$.

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

$$T(1)$$

$$\begin{array}{c} \Theta(n) \\ \swarrow \quad \searrow \\ T\left(\frac{n}{2}\right) \quad T\left(\frac{n}{2}\right) \end{array}$$



$$\begin{array}{c} cn \\ \swarrow \quad \searrow \\ \frac{cn}{2} \quad \frac{cn}{2} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ T\left(\frac{n}{4}\right) \quad T\left(\frac{n}{4}\right) \quad T\left(\frac{n}{4}\right) \quad T\left(\frac{n}{4}\right) \end{array}$$



$$\begin{array}{c} cn \\ \swarrow \quad \searrow \\ \frac{cn}{2} \quad \frac{cn}{2} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \frac{cn}{4} \quad \frac{cn}{4} \quad \frac{cn}{4} \quad \frac{cn}{4} \end{array} \quad \left| \begin{array}{c} cn \quad 0 \\ cn \\ cn \end{array} \right.$$

$$\frac{n}{2^k} = 1$$

$$k = \log_2(n)$$

$$\log_2(n)$$

$$c(\log_2(n) + 1)n = O(n \log(n))$$

n^2 contra $n \lg n$

n	n^2	$n \lg n$
10^3	10^6	$\approx 10^4$
10^6	10^{12}	$\approx 2 \times 10^7$
10^9	10^{18}	$\approx 3 \times 10^{10}$

Lo que significa en la práctica

Con un millón de datos, un algoritmo cuadrático como la burbuja hace un billón de operaciones; el ordenamiento por mezcla, unas veinte millones: la diferencia entre esperar horas y esperar un parpadeo. Partir el problema sí pagó, porque la mezcla aprovecha el trabajo de las dos mitades.

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes**
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

Errores comunes con divide y vencerás

Dónde se caen estos algoritmos

- **Caso base ausente o mal puesto:** sin él, la recursión nunca toca fondo. Es el paso de terminación de esta técnica.
- **Una partición que no reduce:** si una de las partes puede quedar del mismo tamaño que el problema original, el algoritmo se cuelga igual que un ciclo sin progreso.
- **Mezclar olvidando los sobrantes:** cuando una lista se agota, los elementos restantes de la otra deben copiarse; los dos `while` finales de mezclar no son adorno.
- **Equivocar $f(n)$ en la recurrencia:** la mezcla cuesta $\Theta(n)$, no $\Theta(1)$. Con la $f(n)$ errada, el análisis entero da otra cosa.

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python**
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

El costo escondido de `lista[i]`

Una lista de Python no es un arreglo compacto

Guarda referencias a objetos que viven repartidos por la memoria, así que cada `lista[i]` paga una indirección que un arreglo de C o de C++ no paga.

Lo que cambia y lo que no

Ese sobre costo es un factor constante: `lista[i]` sigue siendo $O(1)$, el análisis asintótico no se mueve y por eso el curso trabaja con listas de Python sin remordimiento. Las copias de los slices son otra historia: agregan trabajo $\Theta(n)$ donde la división debía ser gratis, y eso sí cambia la cuenta.

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios**
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

Ejercicios

Ejercicios

Para practicar

- 1 Escriba un algoritmo de divide y vencerás que calcule la *suma* de un arreglo. Plantee su recurrencia y estime su costo.
- 2 Escriba un algoritmo de divide y vencerás que cuente cuántas veces aparece un valor x en un arreglo. Plantee su recurrencia.
- 3 Usando que $b^n = (b^{n/2})^2$ cuando n es par, diseñe un algoritmo para calcular b^n . Plantee su recurrencia $T(n) = T(n/2) + \Theta(1)$ y expanda algunos niveles: ¿a qué costo llega?
- 4 Para el primer ciclo de `mezclar`: demuestre la inicialización y la estabilidad de sus invariantes I_0 e I_1 , y use la terminación para concluir que la mezcla es correcta. La estabilidad se parte en dos casos, según cuál copia aporta el menor; en cada uno, diga qué exige el invariante evaluado en el nuevo valor y por qué el estado que dejó el código lo cumple.

Para practicar en el navegador

Los mismos pasos, con respuesta inmediata

En las notas del curso, mezclar lleva el primer ciclo hasta la demostración completa — inicialización, estabilidad por casos y terminación, con su Teorema 2 — y ordenar dibuja el árbol de llamadas nivel por nivel hasta que aparece el $n \lg n$. Los dos de invariantes de ciclo, sumar y factorial, siguen ahí.

Dónde

<https://cardel.github.io/notasUniversidad/2026-II/AyG%20PUJ/C2/Ejercicios/>

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias**
- 10 Apéndice: los invariantes, demostrados

Referencias

-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, tercera edición, MIT Press, 2009. Secciones 2.3.1–2.3.2, pp. 30–37, y Capítulo 4.
-  J. Kleinberg y É. Tardos. *Algorithm Design*, Addison-Wesley, 2006. Capítulo 5.
-  C. Rocha. *Diseño y Análisis de Algoritmos*.

Contenido

- 1 Repaso: invariantes de ciclo
- 2 Un problema para partir en dos
- 3 El esquema divide y vencerás
- 4 El máximo, ahora recursivo
- 5 Ordenamiento por mezcla
- 6 Errores comunes
- 7 Los costos que esconde Python
- 8 Ejercicios
- 9 Referencias
- 10 Apéndice: los invariantes, demostrados

El repaso en acción: la demostración

Inicialización

Por las líneas 1–2, $\text{ans} = 1$ e $i = 1$: se cumple $1 \leq 1 \leq N + 1$ y $\text{ans} = 1 = 0! = (1 - 1)!$. 

Estabilidad

Se considera una iteración arbitraria $i = j$, diferente a la última (con $j \leq N$: de lo contrario el cuerpo no se ejecuta), y se asume que antes de ella valen $1 \leq j \leq N + 1$ y $\text{ans} = (j - 1)!$. Las líneas 4–5 dejan $\text{ans} = (j - 1)! \cdot j$ e $i = j + 1$. Lo que hay que verificar es que los invariantes, *evaluados en* $j + 1$, describen ese estado. I_1 evaluado en $j + 1$ exige $\text{ans} = ((j + 1) - 1)! = j!$, y el código dejó $(j - 1)! \cdot j$, que es $j!$ por la definición de factorial: coinciden.  I_0 evaluado en $j + 1$ exige $1 \leq j + 1 \leq N + 1$: de $1 \leq j$ sale $1 \leq j + 1$, y de $j \leq N$ sale $j + 1 \leq N + 1$.  Por lo tanto, los invariantes son estables.

El repaso en acción: la demostración

Terminación

Se puede garantizar que el ciclo termina: i sube de 1 en 1 hasta superar a N . Finalmente, al salir, la condición $i \leq N$ es falsa, o sea $i \geq N + 1$; e I_0 acota $i \leq N + 1$. El único valor que cumple las dos cosas es $i = N + 1$. Sustituyéndolo en I_1 : $\text{ans} = (N + 1 - 1)! = N!$, que coincide con la poscondición: los invariantes son correctos. ■

El repaso en acción: el teorema

Teorema 2

$\text{fact}(N)$ para cualquier $N \geq 0$ produce $N!$. *Demostración:* es trivial a partir de la correctitud de I_0 e I_1 (Teorema 1): la terminación deja $i = N + 1$, y sustituirlo en I_1 da $\text{ans} = N!$. ■

Otro ejercicio: Teorema 1 e inicialización

Los invariantes

$$I_0 : 0 \leq i \leq N$$

$$I_1 : \text{ans} = \exists p \in [0..i). A[p] = v$$

ans responde la pregunta del problema, pero solo sobre el prefijo ya revisado $A[0..i)$.

```
0  bool solve(vector<int>& A, int v) {  
1      bool ans = false;  
2      int i = 0;  
3      while (i < A.size()) {  
4          if (A[i] == v)  
5              ans = true;  
6          ++i;  
7      }  
8      return ans;  
9  }
```

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación. El desarrollo queda en el apéndice.

Otro ejercicio: Teorema 1 e inicialización

$$I_0: 0 \leq i \leq N$$

$$I_1: \text{ans} = \exists p \in [0..i). A[p] = v$$

$i \geq 0$

$\exists p \in [0..0), A[p] = v$
 $\emptyset$

```
0  bool solve(vector<int>& A, int v) {  
1      bool ans = false;  
2      int i = 0;  
3      while (i < A.size()) {  
4          if (A[i] == v)  
5              ans = true;  
6          ++i;  
7      }  
8      return ans;  
9  }
```

Inicialización

Por las líneas 1–2, $\text{ans} = \text{false}$ e $i = 0$: se cumple $0 \leq 0 \leq N$, y el existencial sobre el rango vacío $[0..0)$ es falso = ans .



Otro ejercicio: estabilidad por casos y terminación

Estabilidad

Se considera una iteración arbitraria $i = j$, diferente a la última (con $j < N$), y se asumen $0 \leq j \leq N$ y $\text{ans} = \exists p \in [0..j). A[p] = v$. La línea 4 abre dos casos:

- $A[j] = v$: ans pasa a `true`. El existencial sobre $[0..j+1)$ es verdadero porque la propia posición $p = j$ es un testigo: $A[j] = v$, y j es una posición válida del arreglo por $j < N$. ✓
- $A[j] \neq v$: ans conserva su valor. Un testigo del rango extendido $[0..j+1)$ estaría en $[0..j)$ o sería j ; como $A[j] \neq v$, j no sirve, y los testigos posibles son exactamente los de $[0..j)$: el existencial no cambia de valor. ✓

Tras la línea 6, $i = j + 1$; I_0 evaluado en $j + 1$ exige $0 \leq j + 1 \leq N$, que sale de $0 \leq j < N$ sumando 1. Por lo tanto, los invariantes son estables.

Otro ejercicio: estabilidad por casos y terminación

Terminación

Se puede garantizar que el ciclo termina: i sube de 1 en 1 e l_0 lo acota por N . Finalmente, al salir, la condición $i < N$ es falsa, o sea $i \geq N$; e l_0 acota $i \leq N$. El único valor que cumple las dos cosas es $i = N$, y sustituirlo en l_1 da $\text{ans} = \exists p \in [0..N). A[p] = v$: la poscondición. Los invariantes son correctos. ■

Teorema 2

$\text{solve}(A, v)$ produce `true` si v está en A y `false` en caso contrario.
Demostración: es trivial a partir de la correctitud de l_0 e l_1 (Teorema 1): la terminación deja $i = N$, y sustituir ese valor en l_1 entrega la poscondición. ■

Otro ejercicio: estabilidad por casos y terminación

Guárdese este esquema

La estabilidad por casos reaparece hoy mismo: el ciclo de mezclar se demuestra igual, con un caso por cada lista que aporta el menor.